

Discrete-time PID Controller Design in Programmable Logical Controllers

Leonas Balasevicius¹, Asta Kunickaite², Vytis Svajunas Janusevicius³

1) Kaunas University of Technology, Studentu st. 48-324, LT-51367 Kaunas, Lithuania, leonas.balasevicius@ktu.lt

2) Kaunas University of Technology, Studentu st. 48-320, LT-51367 Kaunas, Lithuania, a.kunickaite@etc.lt

3) Kaunas University of Technology, Studentu st. 48-229, LT-51367 Kaunas, Lithuania, vytis.janusevicius@ktu.lt

Abstract – In this paper a discrete-time PID controller is presented for a process with delay control. The discrete-time PID controller design in programmable logic controller has been researched. Finally, the complete algorithm is tested in the programmable logic controller, and the results of this test are explained.

Keywords – Discrete-time systems, time delay systems, PID control, programmable logic controllers, remote control.

I. INTRODUCTION

Control theory is extending to new areas. The idea to extend control loops over the standard industrial networks is being researched widely. The problem is that the network induces a varying delay time into the control loop, which has to be taken into account when designing a controller. The paper investigates discrete-time PID controller design and implementation for systems with delay time, using Matlab software simulation and programmable logical controller (PLC) software.

There are a lot of reasons for the delay time to appear in the process. It can be caused by heating or cooling processes, higher order process approximation with lower order model, liquid flow in long pipes and more. Very often in industrial control the delay time originates between a sensor and an actuator. In such systems the delay time appears because sensors are mounted far away from actuators.

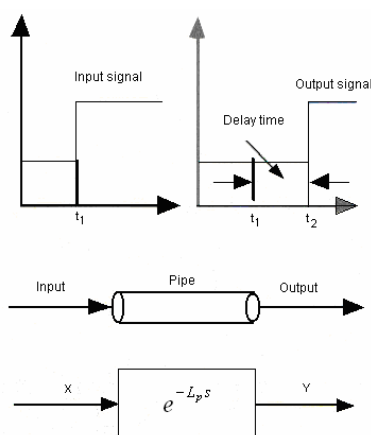


Fig. 1. Delay time representation (L_p -delay time).

This paper further presents discrete-time PID controller design for plant [1]:

$$y(s) = \frac{0.66e^{-2.6s}}{6.7s + 1}u(s) \quad (1)$$

II. DESIGN OF A CONTINUOUS PID CONTROLLER

The controllers that are used to control processes can have several structures. The choice of the structure determines how well the plant can be controlled. The plant poses some guidelines and restrictions on the controller. The choice of a controller determines the best possible performance that is achieved by a well-tuned controller.

Block diagram of a simulated control system is presented below.

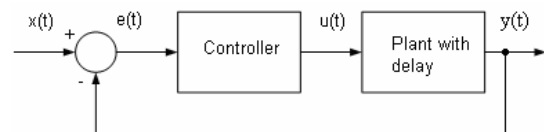


Fig. 2. Block diagram of simulated closed loop system.

A PID controller structure is selected to control the plant (1), as PID controller is used extensively in industry as an all-in-all controller, mostly because it is an intuitive control algorithm.

Unity Pro software [2] for Schneider Electric programmable logical controllers has the function block, which replicates a PID controller with a transfer function:

$$G(s) = gain * \left(1 + \frac{1}{T_i s} + \frac{T_d s}{T_{dlag} s + 1} \right) \quad (2)$$

here - $gain$ – proportional action coefficient; T_i – reset time; T_d – rate time; T_{dlag} – D component delay time.

Controller's output is determined through the discrete PID closed loop algorithm. The sampling time depends on the PLC program cycle, which varies from ms to ms and that is why the discrete-time controller approximates a continuous controller and is tuned as if it was a continuous controller.

By using Matlab Simulink software it is easy to find responses of open and closed loop systems to the unit step input and to design a PID controller with a transfer function:

$$G(s) = P + \frac{I}{s} + Ds \quad (3)$$

here - P – proportional term; I – integral term; D – derivative term.

A PID controller (3) was designed with such parameters: $P=2.6$, $I=0.32$, $D=1.1$, for plant (1). System response to the unit step input, simulated in Matlab environment, is presented in Fig. 3.

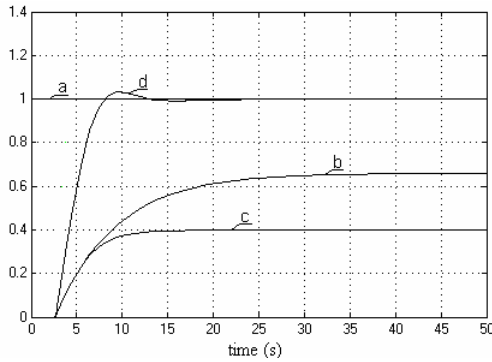


Fig. 3. System response to the unit step input: a – unit step input; b – open loop system response; c – closed loop system with unity negative feedback response; d – closed loop system with a continuous PID controller response.

PID controller parameters, designed in Matlab environment allow calculating the parameters of a PID controller (2) used in Unity Pro software. Correspondence between coefficients in (2) and (3) equations are: $gain=P$, $T_i=P/I$, $T_d=D/P$, where $T_{d\text{lag}}=0$. System response to the unit step input, simulated in PLC environment with Unity Pro software, is presented in Fig. 4.

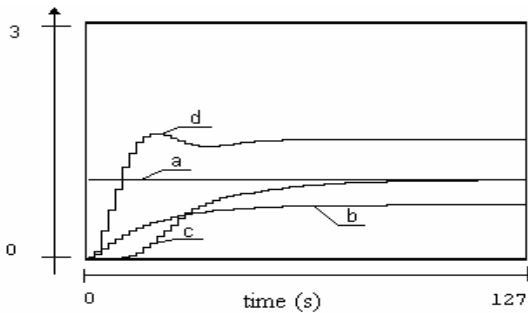


Fig. 4. System response to the unit step input: a – unit step input; b – open loop system response; c – closed loop system with continuous PID controller response; d – PID controller output.

From the system responses, (Fig. 3 and Fig. 4) it can be concluded, that the dynamic characteristics of the process response are changing for the worse, when the parameters of the PID controller (2) are calculated from the PID

controller (3) used in Matlab environment. A PID controller (2) with parameters: $gain=3.5$, $T_i=5.6s$, $T_d=0.9s$, $T_{d\text{lag}}=0$ was designed in Unity Pro software, to achieve better dynamic characteristics of the process response (see Fig.5).

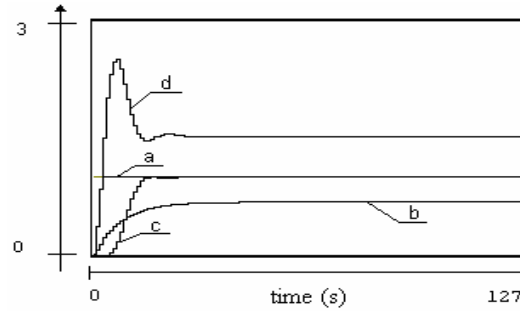


Fig. 5. System response to the unit step input: a – unit step input; b – open loop system response; c – closed loop system with continuous PID controller response; d – PID controller output.

Control system simulation results in Matlab and Unity Pro software environments are presented in Table I.

TABLE I
THE CHARACTERISTICS OF THE CONTROL SYSTEM WITH
CONTINUOUS PID CONTROLLER

System	Characteristic	Overshoot, %	Settling time, s
Open loop system, obtained in Matlab environment		-	37
Open loop system, obtained in Unity Pro environment		-	46
Closed loop system with a PID controller ($P=2.6$, $I=0.32$, $D=1.1$) designed in Matlab environment		3,5	20
Closed loop system with a PID controller ($gain=2.6$, $T_i=8.125s$, $T_d=0.423s$) designed in Unity Pro environment		0	115
Closed loop system with a PID controller ($gain=3.5$, $T_i=5.6s$, $T_d=0.9s$) designed in Unity Pro environment		0	25
Closed loop system with a PID controller ($P=3.5$, $I=0.625$, $D=3.15$) designed in Matlab environment		21	25

It can be concluded, that in Matlab environment, it is possible to design a PID controller with a bigger system overshoot than required, because in Unity Pro environment, the system overshoot will be lesser than in Matlab environment.

III. DESIGN OF A DISCRETE-TIME PID CONTROLLER

There are some methods to obtain a discrete transfer function from a continuous transfer function. This paper investigates a discrete-time PID transfer function

$$G(z) = \frac{q_0 + q_1 z^{-1} + q_2 z^{-2}}{1 - z^{-1}} \quad (4)$$

calculation and implementation in PLC software - Unity Pro. Here coefficients q_i are derived from equations [3]:

$$\begin{aligned} q_0 &= gain * \left(1 + \frac{T_d}{T}\right) \\ q_1 &= -gain * \left(1 + \frac{2 * T_d}{T} - \frac{T}{T_i}\right) \\ q_2 &= gain * \frac{T_d}{T} \end{aligned} \quad (5)$$

here T – sample time.

The programmable logical controller does calculations in discrete-time. In distributed control systems, a discrete-time version of the controller is required because the measurements and the control signal are sent over the network as packets.

The discrete-time PID controller (4) was implemented in Unity Pro software as a function block, by using the algorithm (see Fig. 6) of the Transpose-Direct-Form II Biquad digital filter [4].

Discrete-time PID controller (4) was programmed with Unity Pro software using ST (Structured text, similar to the Pascal programming language) language. It is necessary to pay attention to the fact that it was very important to choose the correct execution order of the blocks (see Fig. 6) for the discrete-time PID controller to work correctly.

The fundamental concepts of PLC programming are common to all manufacturers, so the discrete-time PID controller algorithm, that is presented here, is interchangeable between different PLC manufacturers.

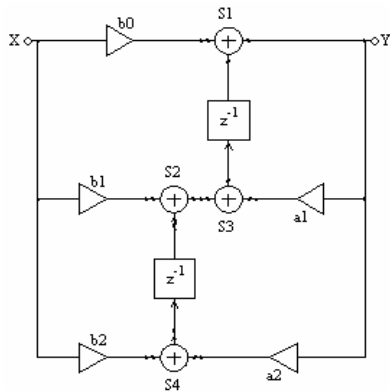


Fig. 6. Block diagram of Transpose-Direct-Form II Biquad digital filter

The research of a designed discrete-time PID controller (4) and a plant (1), was done using the scheme presented in Fig. 7. A plant model was implemented in the PLC2 (Quantum CPU 650 50). PLC1 (Quantum CPU 650 50) had a program which implemented a designed discrete-time PID controller and was responsible for communications between PLC1 and PLC2. Communications between the PLCs were carried out in

two different ways: using the deterministic Modbus Plus network (every 100 ms) or a non-deterministic Ethernet TCP/IP (Modbus TCP) network.

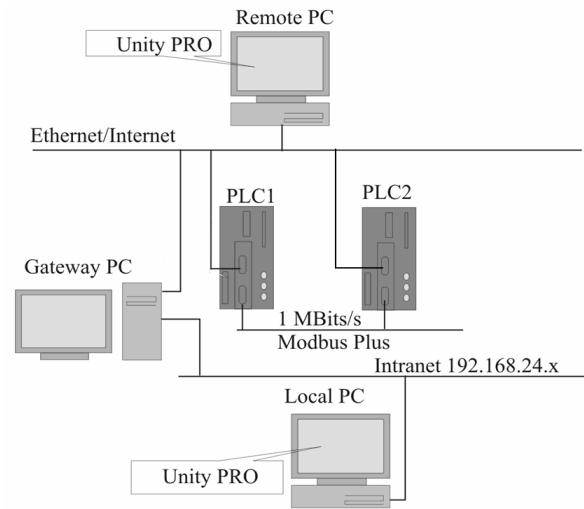


Fig. 7. A discrete-time PID controller and a plant research scheme.

A computer with Unity Pro software was connected to the PLC1 through the Ethernet, to supervise program states and to draw needed graphics.

System response to the unit step input has been researched. The parameters of the discrete-time PID controller are calculated, using the parameters of the PID controller, designed in Matlab environment. Communications between the PLCs were carried out using the deterministic Modbus Plus network.

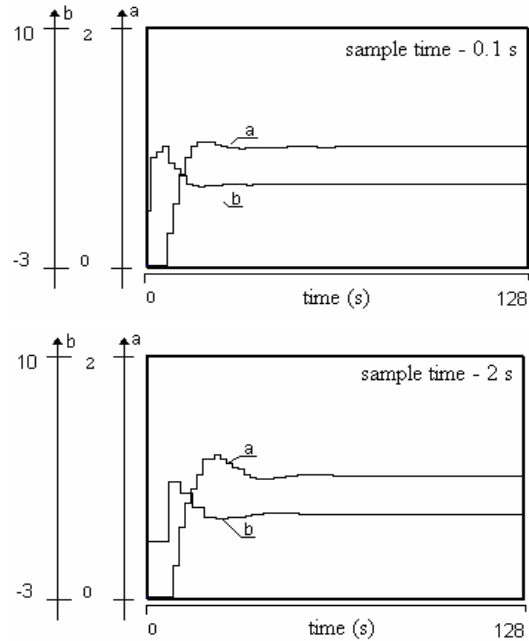


Fig. 8. Responses to the unit step input: a – closed loop system with a discrete-time PID controller; b – discrete-time PID controller output.

System response to the unit step disturbance, which was applied to the system output, has been simulated (see Fig. 9).

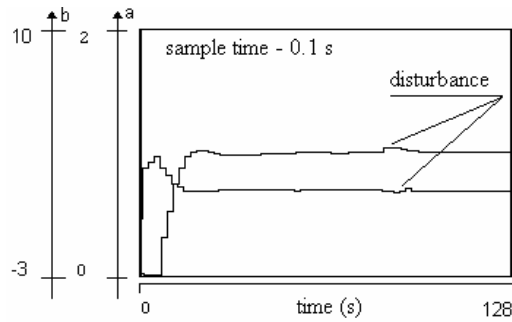


Fig. 9. Responses to the unit step disturbance: a – closed loop system with a discrete-time PID controller; b – discrete-time PID controller output.

The parameters of the discrete-time PID controller are calculated, using the parameters of the PID controller, designed in Unity Pro environment. Communications between the PLCs were carried out using the deterministic Modbus Plus network. System response to the unit step input is presented in Fig. 10.

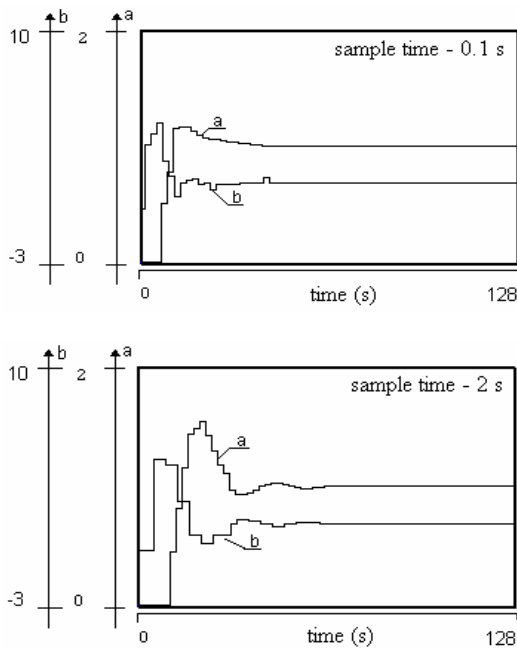


Fig. 10. Responses to the unit step input: a – closed loop system with a discrete-time PID controller; b – discrete-time PID controller output.

System response to the unit step disturbance, which was applied to the system output, has been simulated (see Fig. 11).

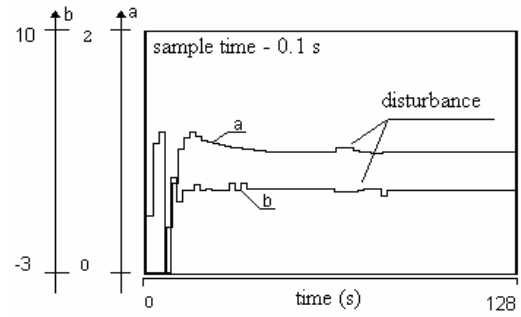


Fig. 11. Responses to the unit step disturbance: a – closed loop system with a discrete-time PID controller; b – discrete-time PID controller output.

Simulation results of the control system in Quantum PLCs that are connected through the deterministic Modbus Plus network are presented in Table II.

TABLE II
THE CHARACTERISTICS OF THE CONTROL SYSTEM WITH A DISCRETE-TIME PID CONTROLLER (DETERMINISTIC NETWORK)

Sample time, s	Overshoot, %		Settling time, s	
	The parameters of the discrete-time PID controller are calculated from		The parameters of the discrete-time PID controller are calculated from	
	Matlab environment	Unity Pro environment	Matlab environment	Unity Pro environment
0.1	4.5	13	27	40
0.3	4.5	22	27	40
0.5	4.5	22	27	40
0.7	8.2	26	39	45
1	9	35	43	52
2	18	52	50	56

It can be concluded that the increasing sample time influences system overshoot and settling time. The discrete-time PID controller compensates the disturbance well enough. The discrete-time PID controller turned out to work best when its parameters are calculated, using the parameters of the PID controller, designed in Matlab environment. It performs equally enough with a short enough sampling time, with the expense of a more active control signal.

Further on, the system response to the unit step input, when communications between the PLCs were carried out using non-deterministic Ethernet TCP/IP (Modbus TCP) network has been researched. The parameters of the discrete-time PID controller are calculated, using the parameters of the PID controller, designed in Matlab environment.

The system responses are presented in Fig. 12 and Fig. 13.

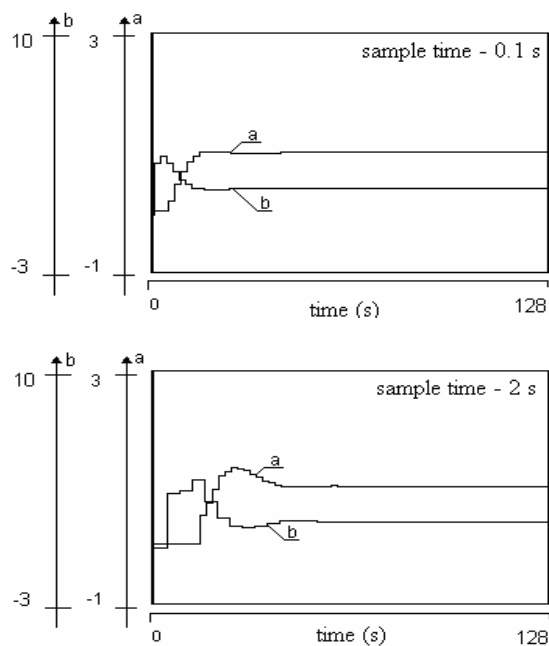


Fig. 12. Responses to the unit step input: a – closed loop system with a discrete-time PID controller; b – discrete-time PID controller output.

The parameters of the discrete-time PID controller are calculated, using the parameters of the PID controller, designed in Unity Pro environment.

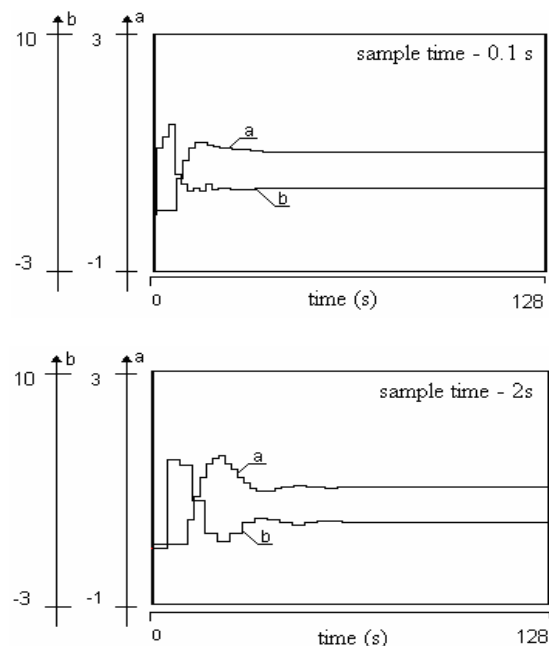


Fig. 13. Responses to the unit step input: a – closed loop system with a discrete-time PID controller; b – discrete-time PID controller output.

Simulation results of the control system in Quantum PLCs that are connected through non-deterministic

Ethernet TCP/IP (Modbus TCP) network are presented in Table III.

TABLE III
THE CHARACTERISTICS OF THE CONTROL SYSTEM WITH A DISCRETE-TIME PID CONTROLLER (NON-DETERMINISTIC NETWORK)

Sample time, s	Overshoot, %		Settling time, s	
	The parameters of the discrete-time PID controller are calculated from		The parameters of the discrete-time PID controller are calculated from	
	Matlab environment	Unity Pro environment	Matlab environment	Unity Pro environment
0.1	1	18	27.2	39
0.3	2	22	27.2	39
0.5	4	26	27.2	39
0.7	26	33	38.8	39
1	54	37	38.8	40
2	33	54	38.9	58

It can be concluded that the discrete-time PID controller turned out to work best when sampling time is short enough. The settling time is less influenced by sample time, when communications between the PLCs were carried out using non-deterministic Ethernet TCP/IP (Modbus TCP) network.

IV. CONCLUSIONS

There has been a study on a discrete-time PID controller design in programmable logic controller. Simulation tests with the Schneider Electric PLC platform have been performed. The following observations have been made based on the results:

- digital filters can be used in the PLC to implement a discrete transfer function;
- the discrete-time PID controller turned out to work best when its parameters are calculated, using the parameters of the PID controller, designed in Matlab environment;
- the discrete-time PID controller can be used to control a process with varying delay time, as long as the variation in the delay is small compared to the sample time;
- a discrete-time PID controller function block can be obtained for general use in other programmable logic controller systems.

REFERENCES

- [1] Babatunde A. Ogunnaike. *Process Dynamics, Modelings, and Control*. New York, Oxford, Oxford University Press, 1994. p. 800.
- [2] *Unity Electronic Documentation*, Vol. 2, Telemecanique, 2005.
- [3] R. Isermann. *Digitale Regel-systeme. Zweite, uberarbeitete und erweiterte Auflage. Band I*, Grundlagen Deterministische Regelungen, Springer-Verlag, Berlin, 1998.
- [4] M. D. Lutovac, D. V. Tosic, B. L. Evans. *Filter Design for Signal Processing Using MATLAB and Mathematica*. Prentice Hall, Upper Saddle River, New Jersey. 2001.